BANCO PICHINCHA

Caso de Uso

Proceso de Selección: Ingeniero de Datos Senior

Nombre del Caso	**Pipeline Inteligente de Aplanado y Monitoreo de Logs desde APIs**
Tiempo Estimado	1-2 días de desarrollo + documentación
Fecha de Entrega	Antes del 11 de Mayo de 2026 (Entrevista Técnica)
Modalidad	Documento de diseño + Prototipo funcional (código)

Acuerdo de Confidencialidad

Este documento contiene información confidencial de Banco Pichincha. Al recibirlo, el candidato se compromete a:

- No divulgar el contenido del caso a terceros.
- No publicar la solución en repositorios públicos.
- Eliminar toda la documentación relacionada una vez finalizado el proceso.

1. Contexto del Negocio

El Chapter de Ingeniería de Datos de Banco Pichincha gestiona actualmente más de **4,000 procesos de extracción, transformación y carga (ETL)** dentro del Data Fabric Stratio. Estos procesos integran datos desde múltiples fuentes, incluyendo bases de datos relacionales, archivos y **APIs REST** que exponen información en formato JSON con estructuras anidadas de múltiples niveles.

Uno de los principales desafíos identificados es la **falta de un proceso genérico y estandarizado** para:

- Consumir datos desde APIs con diferentes tipos de autenticación (API Key, OAuth 2.0, Bearer Token, Basic Auth).
- Aplanar estructuras JSON de n-niveles de profundidad de forma dinámica.

- Generar métricas e indicadores de monitoreo en tiempo real sobre la ejecución de los pipelines.
- Documentar de forma automatizada los metadatos del proceso para facilitar el traspaso a Operaciones.

Dolor Actual

Cada vez que se requiere integrar una nueva API, los ingenieros desarrollan soluciones ad-hoc, lo que genera:

- Código duplicado y difícil de mantener.
- Falta de visibilidad sobre el estado de las ingestas.

2. Objetivo del Caso

Diseñar e implementar un **prototipo funcional** de un pipeline genérico que permita:

1. **Consumir datos desde una API REST** con soporte para múltiples tipos de autenticación.
2. **Aplanar dinámicamente** estructuras JSON de cualquier nivel de profundidad.
3. **Persistir los datos** siguiendo una arquitectura propuesta
4. **Generar métricas de monitoreo** que alimenten un panel de indicadores básico.
5. **Documentar el proceso** de forma que facilite su traspaso a Operaciones.

Entregables Esperados

1. **Documento de Diseño:** Arquitectura, diagrama de flujo, decisiones técnicas, modelo de datos.
2. **Código Funcional:** Pipeline en Python/PySpark orquestado con Airflow.
3. **Documentación:** documento técnico de la propuesta.
4. **Panel de Métricas:** Visualización básica de indicadores (puede ser un notebook, un CSV, o un dashboard simple).

3. Requisitos Funcionales

3.1. Consumo de API REST

Requisito	Descripción
RF-01: Múltiples autenticaciones	El pipeline debe soportar al menos 2 tipos de autenticación: API Key (en header o query param) y Bearer Token. El tipo de autenticación debe ser configurable sin modificar el código.

RF-02: Paginación	Debe manejar APIs paginadas (offset/limit o cursor-based) para extraer todos los registros disponibles.
RF-03: Manejo de errores	Implementar reintentos con backoff exponencial ante errores HTTP 429 (rate limit) y 5xx. Registrar todos los errores en un log estructurado.
RF-04: Configuración externa	Los parámetros de conexión (URL, headers, credenciales, paginación) deben leerse desde parámetros o un archivo de configuración (YAML/JSON).

3.2. Aplanado Dinámico de JSON

Requisito	Descripción
RF-05: N-niveles de profundidad	El proceso debe aplanar estructuras JSON de cualquier profundidad de forma recursiva, generando columnas con nomenclatura `nivel1_nivel2_nivel3`.
RF-06: Manejo de arreglos	Los arreglos dentro del JSON deben expandirse (explotar) generando múltiples filas, o bien almacenarse como tipo array según configuración.
RF-07: Detección de esquema	El proceso debe inferir el esquema del JSON y generar un archivo de metadatos con la estructura detectada (nombre, tipo, profundidad).

3.3. Persistencia y Arquitectura

Requisito	Descripción
RF-08: Persistencia	Persistir el JSON crudo tal como viene de la API, particionado por fecha de ingesta, en formato Parquet o JSON comprimido.

RF-09: Persistencia	Persistir la versión aplanada con tipado correcto, deduplicación y columnas de auditoría (`fecha_ingesta`, periodo, `fuente`, `version_proceso`).
RF-10: Estándar de rutas S3	Las rutas deben seguir el estándar definido por Arquitectura de datos

3.4. Métricas y Monitoreo

Requisito	Descripción
RF-11: Métricas de ejecución	Registrar: tiempo de ejecución, registros procesados, registros fallidos, tamaño de datos, número de reintentos.
RF-12: Tabla de métricas	Persistir las métricas en una tabla de control que permita análisis histórico y tendencias.
RF-13: Alertas básicas	Definir umbrales y generar alertas (log/notificación) cuando: tasa de error > 5%, tiempo de ejecución > 2x promedio histórico.

4. Requisitos No Funcionales

Código	Categoría	Descripción
RNF-01	Rendimiento	El pipeline debe procesar al menos 10,000 registros JSON en menos de 5 minutos en un ambiente local o de desarrollo.
RNF-02	Escalabilidad	El diseño debe contemplar escalamiento horizontal (más workers en Spark/Airflow) sin cambios en el código.

RNF-03	Mantenibilidad	Código modular, documentado con docstrings, y siguiendo PEP-8. Funciones con responsabilidad única.
RNF-04	Seguridad	Las credenciales nunca deben estar en código. Usar variables de entorno o gestores de secretos.
RNF-05	Observabilidad	Logs estructurados (JSON) con niveles INFO, WARNING, ERROR. Incluir identificador de ejecución (run_id).
RNF-06	Documentación	README completo, diagrama de arquitectura, diccionario de datos generado automáticamente.

5. Stack Tecnológico Sugerido

El candidato debe utilizar las herramientas del ecosistema de Banco Pichincha. Se permite flexibilidad en la implementación del prototipo:

Categoría	Tecnología Requerida	Alternativa para Prototipo
Lenguaje	**Python 3.9+**	—
Procesamiento	**PySpark**	Pandas (para prototipo local)
Orquestación	**Apache Airflow**	Airflow local (Docker) o script con scheduler básico
Almacenamiento	**S3 / Data Fabric Stratio**	Sistema de archivos local o MinIO
Formato de datos	**Parquet**	—

Configuración	**Parámetros**	**YAML / JSON**
Visualización	**PowerBI**	Jupyter Notebook, Streamlit, o CSV + Excel
Versionamiento	**Git (Azure DevOps)**	GitHub, GitLab

6. Arquitectura de Referencia

El candidato debe proponer una arquitectura detallada que incluya:

- Componentes del pipeline y su interacción.
- Flujo de datos desde la fuente hasta las zonas de destino.
- Manejo de errores y reintentos.
- Generación y almacenamiento de métricas.
- Estrategia de particionamiento y compresión.

7. API de Prueba

Para el desarrollo del prototipo, el candidato puede utilizar cualquiera de las APIs públicas que contienen estructuras JSON anidadas o Apis de las diferentes fuentes a nivel de Banco.

> Nota Importante
> El candidato debe demostrar que su solución es **genérica**, no específica para una API. Durante la entrevista técnica, se solicitará configurar el pipeline para consumir una API diferente sin modificar el código fuente.

8. Criterios de Evaluación

La Prueba de Concepto será evaluada considerando los siguientes criterios:

Criterio	Peso	Descripción
Diseño de Arquitectura	**25%**	Claridad del diseño, aplicación correcta de arquitectura Medallion, modularidad, escalabilidad del diseño propuesto.

Calidad del Código	**25%**	Código limpio, buenas prácticas (PEP-8), manejo de errores, uso de patrones de diseño, cobertura de pruebas unitarias.
Genericidad de la Solución	**20%**	Capacidad de adaptar el pipeline a diferentes APIs sin modificar código. Flexibilidad en configuración.
Documentación	**15%**	README completo, diagramas claros, documentación en código, plantilla de traspaso a Operaciones.
Métricas y Observabilidad	**15%**	Implementación de métricas, logs estructurados, visualización de indicadores, alertas básicas.

Puntuación mínima requerida

Para considerar la Prueba de Concepto como **aprobada**, el candidato debe obtener al menos **60 puntos sobre 100** en la evaluación global, sin ningún criterio individual por debajo de **50%**.

9.1. Rúbrica de Calificación

Nivel	Puntos	Descripción General
Excelente	90-100	Supera las expectativas. Solución elegante, bien documentada, con consideraciones de producción. Demuestra visión de arquitecto.
Bueno	75-89	Cumple todos los requisitos. Código limpio, documentación adecuada. Algunas áreas de mejora menores.
Aceptable	60-74	Cumple los requisitos mínimos. Funcionalidad correcta pero con oportunidades de mejora en diseño o documentación.
Insuficiente	40-59	No cumple algunos requisitos importantes. Código difícil de mantener o documentación incompleta.

No Aceptable	0-39	No cumple los requisitos mínimos. Solución incompleta o no funcional.

10. Preguntas para la Defensa

Durante la Entrevista Técnica, el candidato deberá defender su solución. Algunas preguntas que puede esperar:

1. **Arquitectura:** ¿Por qué elegiste esta estructura de módulos? ¿Cómo escalaría esta solución a 1,000 APIs simultáneas?
2. **Genericidad:** Muéstrame cómo configuraría el pipeline para consumir [API diferente]. ¿Qué cambiarías?
3. **Aplanado:** ¿Cómo manejas colisiones de nombres al aplanar? ¿Qué pasa con arreglos de objetos?
4. **Errores:** ¿Qué sucede si la API está caída durante 2 horas? ¿Cómo recuperas la ejecución?
5. **Métricas:** ¿Qué indicadores propondrías para medir la salud del Chapter de Ingeniería de Datos?
6. **Traspaso:** ¿Qué información necesita el equipo de Operaciones para monitorear este pipeline?
7. **Mejoras:** Si tuvieras 2 semanas más, ¿qué funcionalidades agregarías?

11. Documentación

Como parte de los entregables, el candidato debe proponer una plantilla de documentación. A continuación, se presenta una estructura sugerida:

Sección	Contenido Esperado
1. Información General	Nombre del proceso, responsable, fecha de creación, versión, criticidad (Alta/Media/Baja).
2. Descripción Funcional	¿Qué hace el proceso? ¿Qué problema resuelve? ¿Quiénes son los usuarios/consumidores de los datos?
3. Arquitectura Técnica	Diagrama de componentes, flujo de datos, tecnologías utilizadas, dependencias.

4. Fuentes de Datos	Origen de los datos, credenciales (referencia a secretos), frecuencia de actualización.
5. Modelo de Datos	Diccionario de datos con nombre de campo, tipo, descripción, valores permitidos.
6. Configuración	Variables de entorno, archivos de configuración, parámetros ajustables.
7. Monitoreo y Alertas	Métricas clave, umbrales de alerta, contactos de escalamiento, SLAs.
8. Procedimientos Operativos	Cómo iniciar/detener, cómo re-ejecutar, cómo revisar logs, resolución de errores comunes.
9. Historial de Cambios	Registro de versiones, cambios realizados, autor, fecha.

12. Cronograma

Fecha	Actividad	Responsable
29 de Abril	Entrega del caso de uso al candidato	Comité de Selección
29 Abr — 10 May	Desarrollo de la Prueba de Concepto	Candidato
10 de Mayo (23:59)	Fecha límite de entrega del repositorio	Candidato

11 de Mayo	Entrevista Técnica — Defensa de la Prueba de Concepto	Candidato + Panel

Entrega de la propuesta

El candidato debe enviar toda la documentación antes del **10 de Mayo a las 23:59** al correo del de los Chapter Leads de Ingeniería de Datos. Se considerará la última versión disponible antes de la fecha límite.

13. Contacto para Consultas

Durante el desarrollo de la Prueba de Concepto, el candidato puede enviar consultas de aclaración a:

Canal de consultas	Ramiro Merchán – Carlos Medina
Tiempo de respuesta	Máximo 24 horas hábiles
Tipo de consultas permitidas	Aclaraciones sobre requisitos. **No** se proporcionará orientación sobre implementación.